LAPORAN ETS
METODE OPTIMASI METAHEURISTIK



Disusun oleh :

Jason Alexander Widodo	5052241001
Constantine Calvin Gosal Jo	5052241010
Gilbert Emanuel Sambira	5052241018
Affriza Wildan Fauzan	5052241033
Dimas Rizki Gemilang Bajuri	5052241040

Dosen Pengampu :

Dr. Irhamah, S.Si., M.Si.

PROGRAM STUDI S1 SAINS DATA
DEPARTEMEN STATISTIKA
FAKULTAS SAINS DAN ANALITIKA DATA
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA

2026

DAFTAR ISI

DAFTAR ISI ..... **ii**
DAFTAR TABEL ..... **iv**
DAFTAR GAMBAR ..... **iv**
BAB I **1**
PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Penelitian 1
1.4 Batasan Masalah 2
BAB II **3**
TINJAUAN PUSTAKA **3**
2.1 Travelling Salesman Problem (TSP) 3
2.2 TSP Simetris dan Asimetris 3
2.3 Jarak Euclidean 3
2.4 Metode Konstruktif (Contructive Heuristic) 3
2.4.1 Nearest Neighbor 3
2.4.2 Nearest Insertion 4
2.4.3 Farthest Neighbor 4
2.4.4 Arbitary Solution 4
2.5 Metode Local Search 4
2.5.1 2-opt 4
2.6 Metaheuristik 5
2.6.1 Tabu Search 5
2.6.2 Simulated Annealing 5
2.7 Pengukuran Kinerja 5
2.8 Teknologi yang Digunakan 6
BAB III **7**
ANALISIS, PERANCANGAN, DAN IMPLEMENTASI SISTEM **7**
3.1 Metodologi Penelitian 7
3.2 Teknologi Implementasi 7
3.3 Perancangan Arsitektur Sistem 7
3.4 Perancangan Struktur Data 8
3.5 Perancangan Algoritma 8
3.5.1 Nearest Neighbor 8
3.5.2 Nearest Insertion 9
3.5.3 Farthest Neighbor 9
3.5.4 Arbitrary Solution 9
3.5.5 2-opt 9
3.5.6 Tabu Search 10
3.5.7 Simulated Annealing 10
3.6 Perancangan Antarmuka Pengguna 11
3.7 Metrik Evaluasi 11
3.8 Pengujian Sistem 11

BAB IV **13**
HASIL DAN PEMBAHASAN **13**
4.1 Rancangan Eksperimen 13
4.1.1 Pengujian kombinasi metode 13
4.1.2 Pengujian perubahan parameter metaheuristik 14
4.2 Hasil pada Data Lima Kota 14
4.3 Perbandingan Solusi Awal 15
4.4 Perbandingan Metode Perbaikan 15
4.5 Pengaruh Solusi Awal 16
4.6 Pengaruh Perubahan Parameter Metaheuristik 16
4.6.1 Pengaruh Maksimum Iterasi Tabu Search 17
4.6.2 Pengaruh Tabu Tenure 17
4.6.3 Pengaruh Temperatur Awal Simulated Annealing 18
4.6.4 Pengaruh Cooling Rate Simulated Annealing 18
4.6.5 Pengaruh Iterasi per Temperatur 19
4.6.6 Perbandingan Hasil Parameter 19
BAB V **20**
KESIMPULAN DAN SARAN **20**
5.1 Kesimpulan 20
5.2 Saran 20
DAFTAR PUSTAKA **21**
LAMPIRAN **22**

DAFTAR TABEL

Tabel 3.1 Skenario Pengujian Sistem....12
Tabel 4.1 Parameter Tetap Eksperimen Pertama....14
Tabel 4.2 Parameter Uji Eksperimen Kedua....14
Tabel 4.3 Hasil Metode Konstruksi dan Metode Perbaikan pada 5 Titik....15
Tabel 4.4 Perbandingan Solusi Awal Metode Perbaikan....15
Tabel 4.5 Perbandingan Solusi Metode Perbaikan....16
Tabel 4.6 Perbandingan Solusi Perubahan Maksimum Iterasi Tabu Search....17
Tabel 4.7 Perbandingan Solusi Perubahan Maksimum Iterasi Tabu Search....17
Tabel 4.8 Perbandingan Solusi Perubahan Temperatur Awal Simulated Annealing....18
Tabel 4.9 Perbandingan Solusi Perubahan Cooling Rate Simulated Annealing....19
Tabel 4.10 Perbandingan Solusi Perubahan Iterasi per Temperatur Simulated Annealing....19

DAFTAR GAMBAR

Gambar 3.1 Diagram Alur Metodologi Penelitian....7
Gambar 3.2 Diagram Alur Utama Sistem....8

BAB I
PENDAHULUAN

1.1 Latar Belakang

Travelling Salesman Problem (TSP) merupakan salah satu permasalahan optimasi kombinatorial yang paling klasik dan banyak dipelajari dalam bidang ilmu komputer dan riset operasi. Permasalahan ini bertujuan untuk menemukan rute terpendek yang mengunjungi setiap kota tepat satu kali dan kembali ke kota asal. Meskipun formulasi masalahnya sederhana, TSP tergolong ke dalam permasalahan NP-Hard, yang berarti tidak ada algoritma yang dapat menyelesaikan permasalahan ini dalam waktu polinomial untuk ukuran input yg besar.

TSP memiliki banyak aplikasi praktis dalam kehidupan sehari-hari, seperti perencanaan rute pengiriman barang, optimasi jalur distribusi, penjadwalan kunjungan sales, perencanaan rute wisata, dan lain sebagainya. Dalam konteks logistik dan distribusi, efisiensi rute sangat berpengaruh terhadap biaya operasional, waktu tempuh, dan konsumsi bahan bakar. Oleh karena itu, diperlukan metode yang mampu menghasilkan solusi mendekati optimal dalam waktu komputasi yang wajar.

Secara umum, pendekatan penyelesaian TSP dibagi menjadi dua kategori utama, yaitu metode eksak dan metode heuristic/metaheuristik. Metode eksak seperti Branch and Bound dan Dynamic Programming mampu menghasilkan solusi optimal, namun tidak praktis untuk jumlah kota yang besar karena kompleksitas waktunya yang eksponensial. Oleh karena itu, metode heuristic dan metaheuristik menjadi pilihan yang lebih realistis untuk permasalahan berskala besar.

Metode heuristic konstruktif seperti Nearest Neighbor, Nearest Insertion, Farthest Neighbor, dan Arbitrary Solution digunakan untuk membangun solusi awal dengan cepat. Solusi awal ini kemudian diperbaiki menggunakan metode perbaikan seperti 2-opt serta metaheuristik seperti Tabu Search dan Simulated Annealing. Kombinasi antara metode konstruksi dan metode perbaikan diharapkan mampu menghasilkan solusi yang berkualitas dengan waktu komputasi yang efisien.

Berdasarkan latar belakang tersebut, penelitian ini bertujuan untuk mengimplementasikan dan menganalisis kinerja berbagai kombinasi metode konstruksi dan metode perbaikan dalam menyelesaikan permasalahan TSP. Sistem diimplementasikan dalam bentuk aplikasi berbasis web agar dapat digunakan secara interaktif, dengan visualisasi langkah-langkah algoritma dan perbandingan hasil antar metode.

1.2 Rumusan Masalah

Rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana mengimplementasikan algoritma konstruksi Nearest Neighbor, Nearest Insertion, Farthest Neighbor, dan Arbitrary Solution dalam menyelesaikan permasalahan TSP?
2. Bagaimana mengimplementasikan metode perbaikan 2-opt, Tabu Search, dan Simulated Annealing untuk meningkatkan kualitas solusi awal?
3. Bagaimana perbandingan kualitas solusi dan waktu komputasi antara berbagai kombinasi metode konstruksi dan metode perbaikan pada dataset dengan ukuran yang berbeda?
4. Bagaimana pengaruh kualitas solusi awal terhadap hasil akhir yang diperoleh setelah proses perbaikan?
5. Bagaimana merancang aplikasi berbasis web yang mampu memvisualisasikan proses optimasi rute TSP secara bertahap?

1.3 Tujuan Penelitian

Tujuan yang ingin dicapai dalam penelitian ini adalah:

1. Mengimplementasikan empat metode konstruksi, yaitu Nearest Neighbor, Nearest Insertion, Farthest Neighbor, dan Arbitrary Solution untuk membangun solusi awal TSP.

2. Mengimplementasikan tiga metode perbaikan, yaitu 2-opt, Tabu Search, dan Simulated Annealing untuk memperbaiki solusi awal.
3. Membandingkan kinerja setiap kombinasi metode berdasarkan kualitas solusi (biaya rute) dan waktu komputasi.
4. Menganalisis pengaruh solusi awal terhadap kualitas solusi akhir yang dihasilkan
5. Membangun aplikasi berbasis web yang dapat memvisualisasikan proses optimasi TSP secara interaktif dan menyajikan hasil perbandingan antar metode.

1.4 Batasan Masalah

Ditetapkan batasan masalah sebagai berikut:

1. Permasalahan TSP yang diselesaikan adalah TSP simetris pada bidang dua dimensi (koordinat kartesian).
2. Jarak antar titik dihitung menggunakan rumus jarak Euclidean.
3. Data yang digunakan terdiri dari data bawaan sistem (5 kota) serta data koordinat acak berukuran 25, 50, dan 100 titik.
4. Metode konstruksi yang digunakan terbatas pada Nearest Neighbor, Nearest Insertion, Farthest Neighbor, dan Arbitrary Solution.
5. Metode perbaikan yang digunakan terbatas pada 2-opt, Tabu Search, dan Simulated Annealing.
6. Parameter algoritma metaheuristik ditetapkan secara tetap sesuai dengan rancangan eksperimen.
7. Aplikasi diimplementasikan sebagai aplikasi berbasis web dan tidak membahas aspek keamanan serta skalabilitas sistem secara mendalam.

BAB II
TINJAUAN PUSTAKA

2.1 Travelling Salesman Problem (TSP)

Travelling Salesman Problem (TSP) adalah permasalahan optimasi kombinatorial yang bertujuan untuk menemukan rute terpendek yang mengunjungi sekumpulan kota tepat satu kali dan kembali ke kota awal. Secara formal, diberikan himpunan kota $V = \{v_1, v_2, \dots, v_n\}$ dan matriks jarak $D = [d_{ij}]$ yang menyatakan jarak antara kota i dan kota j. TSP bertujuan untuk menemukan permutasi $\boldsymbol{\pi}$ dari kota-kota tersebut yang meminimalkan total jarak:

$$f(\boldsymbol{\pi}) = \sum_{i=1}^{n} d_{\boldsymbol{\pi}(i),\boldsymbol{\pi}(i+1)} + d_{\boldsymbol{\pi}(n),\boldsymbol{\pi}(1)}$$

dengan $\boldsymbol{\pi}(n+1) = \boldsymbol{\pi}(1)$

TSP tergolong sebagai permasalahan NP-Hard, yang berarti kompleksitas waktunya tumbuh secara eksponensial seiring bertambahnya jumlah kota. Untuk n kota, terdapat (n−1)!/2 kemungkinan rute yang berbeda pada TSP simetris. Oleh karena itu, pendekatan eksak hanya praktis untuk jumlah kota yang relatif kecil, sehingga pendekatan heuristic dan metaheuristik menjadi alternatif yang banyak digunakan.

2.2 TSP Simetris dan Asimetris

TSP dapat diklasifikasikan menjadi dua jenis berdasarkan sifat matriks jaraknya:

1. TSP Simetris: Jarak antara kota i dan kota j sama dengan jarak antara kota j dan kota i ($d_{ij} = d_{ji}$). Pada TSP simetris, rute dan rute kebalikannya memiliki biaya yang sama.
2. TSP Asimetris: Jarak antara kota i ke kota j tidak sama dengan jarak dari kota j ke kota i ($d_{ij} \neq d_{ji}$). TSP asimetris umumnya terjadi pada jaringan jalan satu arah atau kondisi lalu lintas yang berbeda.

Pada penelitian ini, TSP yang diselesaikan adalah TSP simetris dengan jarak antar titik dihitung menggunakan rumus jarak Euclidean pada bidang dua dimensi.

2.3 Jarak Euclidean

Jarak Euclidean antara dua titik i dan j dengan koordinat (x_i, y_i) dan (x_j, y_j) dihitung menggunakan rumus:

$$d(i,j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Rumus ini digunakan untuk membentuk matriks jarak yang menjadi dasar perhitungan biaya rute pada sistem yang dibangun.

2.4 Metode Konstruktif (Contructive Heuristic)

Metode konstruktif adalah pendekatan heuristic yang membangun solusi secara bertahap dari kondisi awal kosong hingga terbentuk rute lengkap. Metode ini umumnya cepat dan sederhana, tetapi kualitas solusi yang dihasilkan belum tentu mendekati optimal. Solusi yang dihasilkan biasanya digunakan sebagai solusi awal untuk diperbaiki oleh metode lain.

2.4.1 Nearest Neighbor

Nearest Neighbor (NN) adalah algoritma konstruktif yang paling sederhana. Algoritma ini dimulai dari sebuah titik awal, kemudian secara berulang memilih titik yang

belum dikunjungi dengan jarak terdekat dari titik terakhir. Proses ini berlanjut hingga semua titik telah dikunjungi, kemudian rute ditutup dengan kembali ke titik awal.

Kelebihan NN adalah kesederhanaan dan kecepatan komputasinya, yaitu $O(n^2)$. Namun, kelemahannya adalah solusi yang dihasilkan sangat bergantung pada pemilihan titik awal dan sering kali terjebak pada solusi yang jauh dari optimal.

2.4.2 Nearest Insertion

Nearest Insertion (NI) adalah algoritma konstruktif yang dimulai dengan membentuk sub-tour dari dua titik, yaitu titik awal dan titik terdekatnya. Pada setiap iterasi, algoritma memilih titik yang belum dikunjungi yang paling dekat dengan sub-tour yang telah terbentuk, kemudian menyisipkannya pada posisi yang menghasilkan kenaikan biaya terkecil.

Kenaikan biaya penyisipan titik v di antara titik a dan b dihitung dengan:

$$\Delta = d(a,v) + d(v,b) - d(a,b)$$

Algoritma ini umumnya menghasilkan solusi awal yang lebih baik dibandingkan Nearest Neighbor, dengan kompleksitas waktu $O(n^2)$

2.4.3 Farthest Neighbor

Farthest Neighbor (FN) memiliki mekanisme yang mirip dengan Nearest Neighbor, namun pada setiap langkah algoritma memilih titik yang belum dikunjungi dengan jarak terjauh dari titik terakhir. Pendekatan ini bertujuan untuk menyebarkan kunjungan secara merata, meskipun secara umum menghasilkan solusi awal dengan biaya yang lebih tinggi dibandingkan Nearest Neighbor dan Nearest Insertion.

2.4.4 Arbitary Solution

Arbitrary Solution adalah metode konstruksi yang membentuk solusi awal dengan mengacak urutan seluruh titik. Pengacakan dilakukan menggunakan pseudo-random number generator (PRNG). Untuk mendukung reprodusibilitas eksperimen, ditetapkan sebuah seed sehingga urutan acak yang dihasilkan dapat direproduksi. Metode ini umumnya menghasilkan solusi awal dengan biaya tertinggi, namun berguna sebagai pembanding untuk menguji efektivitas metode perbaikan.

2.5 Metode Local Search

Metode perbaikan lokal bekerja dengan melakukan modifikasi terhadap solusi awal secara iteratif untuk mencari solusi yang lebih baik di lingkungannya. Metode ini berhenti ketika tidak ada lagi perbaikan yang dapat dilakukan atau ketika batas iterasi tercapai.

2.5.1 2-opt

Algoritma 2-opt adalah metode perbaikan lokal yang bekerja dengan menghapus dua sisi dari rute, kemudian menghubungkan kembali kedua bagian rute dengan membalik urutan titik pada segmen yang dipilih. Jika perubahan tersebut menghasilkan biaya yang lebih kecil, maka swap diterima.

Secara formal, jika rute memiliki sisi (a,b) dan (c,d), maka 2-opt menggantinya dengan sisi (a,c) dan (b,d), dengan segmen antara b dan c dibalik urutannya. Perubahan biaya dihitung dengan:

$$\Delta = d(a,c) + d(b,d) - d(a,b) - d(c,d)$$

Proses ini diulang hingga tidak ada lagi perbaikan yang dapat dilakukan atau jumlah pass maksimum tercapai. 2-opt memiliki kompleksitas waktu $O(n^2)$ per iterasi dan merupakan metode perbaikan yang efisien untuk TSP.

2.6 Metaheuristik

Metaheuristik adalah metode optimasi tingkat tinggi yang dirancang untuk menjelajahi ruang solusi secara lebih luas, termasuk kemampuan untuk keluar dari optimum lokal. Berbeda dengan metode perbaikan lokal yang hanya menerima perbaikan, metaheuristik dapat menerima solusi yang lebih buruk dengan aturan tertentu untuk menjelajahi ruang solusi yang lebih luas.

2.6.1 Tabu Search

Tabu Search (TS) adalah metaheuristik yang dikembangkan oleh Glover pada tahun 1986. TS memanfaatkan lingkungan solusi 2-opt, namun pada setiap iterasi algoritma memilih kandidat terbaik yang tidak memiliki status tabu. Status tabu diberikan kepada perpindahan yang baru saja dilakukan untuk mencegah terjadinya siklus dan mendorong eksplorasi ke daerah solusi yang belum dikunjungi.

Tabu list menggunakan mekanisme FIFO (First In First Out), di mana algoritma menyimpan pasangan titik yang berhasil melakukan swap. Perpindahan yang bersifat tabu dapat diterima jika menghasilkan solusi yang lebih baik daripada solusi terbaik global yang pernah ditemukan (aspiration criteria). Parameter yang dapat dikonfigurasi pada TS adalah jumlah iterasi dan tabu tenure (lama status tabu).

2.6.2 Simulated Annealing

Simulated Annealing (SA) adalah metaheuristik yang terinspirasi dari proses annealing pada metalurgi, yaitu proses pemanasan dan pendinginan logam secara perlahan untuk mengurangi cacat kristal. SA memilih kandidat 2-opt secara acak. Solusi yang menurunkan biaya selalu diterima, sedangkan solusi yang menaikkan biaya diterima dengan probabilitas:

$$P(terima) = e^{-\Delta/T}$$

dengan Δ adalah kenaikan biaya dan T adalah temperatur saat ini. Temperatur diperbarui secara bertahap menggunakan rumus:

$$T_{baru} = T_{lama} \times \alpha$$

dengan α adalah *cooling rate* $(0 < \alpha < 1)$. Parameter yang dapat dikonfigurasi pada SA meliputi temperatur awal, temperatur minimum, *cooling rate,* dan jumlah iterasi untuk setiap tingkat temperatur.

2.7 Pengukuran Kinerja

Kinerja algoritma TSP diukur menggunakan beberapa metrik tersebut:

1. Biaya Rute (Cost): Total jarak yang ditempuh pada rute yang dihasilkan. Semakin kecil biaya, semakin baik kualitas solusi.
2. Persentase Perbaikan: Mengukur seberapa besar penurunan biaya dari solusi awal ke solusi akhir, dihitung dengan:
3. Waktu Komputasi: Waktu yang dibutuhkan algoritma untuk menyelesaikan proses optimasi, diukur dalam milidetik (ms).

2.8 Teknologi yang Digunakan

Sistem dalam penelitian diimplementasikan sebagai aplikasi berbasis web dengan teknologi pada Bab 3.2.

2.9 Penelitian Terdahulu

Beberapa penelitian terdahulu yang relevan dengan penelitian ini antara lain:

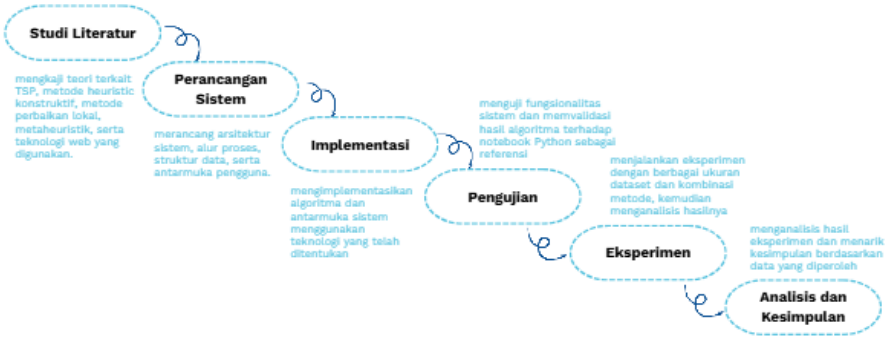
1. Johnson dan McGeoch (1997) melakukan studi komparatif terhadap berbagai metode heuristic dan metaheuristik untuk TSP, termasuk Nearest Neighbor, 2-opt, dan Simulated Annealing. Hasil penelitian menunjukkan bahwa kombinasi solusi awal yang baik dengan metode perbaikan lokal memberikan hasil yang mendekati optimal.
2. Glover (1989) memperkenalkan Tabu Search sebagai metaheuristik yang mampu mengatasi keterbatasan metode perbaikan lokal dengan mekanisme tabu list dan aspiration criteria.
3. Kirkpatrick et al. (1983) memperkenalkan Simulated Annealing sebagai metode optimasi yang terinspirasi dari proses annealing, dengan kemampuan menerima solusi yang lebih buruk untuk keluar dari optimum lokal.
4. Croes (1958) memperkenalkan algoritma 2-opt sebagai metode perbaikan lokal yang efektif untuk TSP, yang hingga kini masih banyak digunakan sebagai dasar metode perbaikan lainnya.

BAB III
ANALISIS, PERANCANGAN, DAN IMPLEMENTASI SISTEM

3.1 Metodologi Penelitian

Penelitian ini menggunakan pendekatan eksperimental yang terdiri dari beberapa tahapan, yaitu:

1. Studi Literatur: mengkaji teori terkait TSP, metode heuristic konstruktif, metode perbaikan lokal, metaheuristik, serta teknologi web yang digunakan.
2. Perancangan Sistem: merancang arsitektur sistem, alur proses, struktur data, serta antarmuka pengguna.
3. Implementasi: mengimplementasikan algoritma dan antarmuka sistem menggunakan teknologi yang telah ditentukan.
4. Pengujian: menguji fungsionalitas sistem dan memvalidasi hasil algoritma terhadap notebook Python sebagai referensi.
5. Eksperimen: menjalankan eksperimen dengan berbagai ukuran dataset dan kombinasi metode, kemudian menganalisis hasilnya.
6. Analisis dan Kesimpulan: menganalisis hasil eksperimen dan menarik kesimpulan berdasarkan data yang diperoleh



Gambar 3.1 Diagram Alur Metodologi Penelitian

3.2 Teknologi Implementasi

Sistem diimplementasikan sebagai aplikasi berbasis web:

- React 19: library JavaScript untuk antarmuka utama.
- Vite 6: build tool dari pembuatan awal aplikasi hingga produk akhir.
- JavaScript: seluruh perhitungan algoritma dijalankan di sisi klien.
- read-excel-file: library untuk membaca data dari file .xlsx.
- Notebook Python: menjadi dasar/panduan awal implementasi setiap algoritma dan pembanding hasil.

Simulasi terjadi pada bidang dua dimensi (koordinat kartesian). Setiap tempat direpresentasikan sebagai titik (x,y), dan jarak dihitung dengan rumus Euclidean.

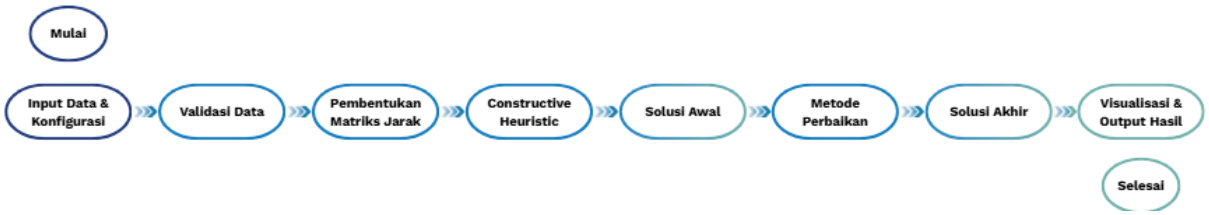
3.3 Perancangan Arsitektur Sistem

Arsitektur sistem dibagi menjadi tiga lapisan utama, yaitu:

1. Lapisan Input Data dan Konfigurasi:
    - Menerima data tempat yang per barisnya mengandung nama tempat dan koordinat (x,y).

- Menerima konfigurasi awal yang terdiri dari metode konstruksi, metode perbaikan, dan parameter algoritma.

2. Lapisan Proses Data:
   - Membentuk matriks jarak berdasarkan koordinat titik
   - Menghasilkan solusi awal menggunakan metode perbaikan (2-opt, Tabu Search, atau Simulated Annealing).
   - Menyimpan setiap langkah algoritma dalam frame simulasi.
3. Lapisan Output Hasil:
   - Menampilkan rute optimal, biaya awal dan akhir, persentase perbaikan, waktu komputasi, grafik perkembangan biaya, pseudocode, dan tabel perbandingan hasil eksekusi.



Gambar 3.2 Diagram Alur Utama Sistem

3.4 Perancangan Struktur Data

Struktur data utama yang digunakan dalam sistem adalah sebagai berikut:

1. Titik (Node): Merepresentasikan sebuah tempat dengan atribut nama, koordinat x, dan koordinat y.
2. Matriks Jarak: Matriks dua dimensi berukuran n×n yang menyimpan jarak Euclidean antar setiap pasangan titik.
3. Rute (Tour): Array yang menyimpan urutan indeks titik yang dikunjungi.
4. Frame Simulasi: Objek yang menyimpan informasi langkah algoritma, meliputi fase proses, rute, biaya, biaya terbaik, metrik iterasi, dan baris pseudocode yang aktif.
5. Tabu List: Struktur FIFO yang menyimpan pasangan titik yang baru saja melakukan swap pada Tabu Search.
6. State Simulated Annealing: Menyimpan temperatur saat ini, temperatur minimum, cooling rate, dan iterasi per temperatur.

3.5 Perancangan Algoritma

3.5.1 Nearest Neighbor

Pseudocode

```
function nearestNeighbor(nodes, start):
    tour = [start]
    unvisited = semua node kecuali start
    current = start
    while unvisited tidak kosong:
        next = node di unvisited dengan jarak terkecil dari current
        tour.append(next)
        unvisited.remove(next)
```

```
        current = next
    return tour
```

3.5.2 Nearest Insertion

Pseudocode

```
function nearestInsertion(nodes, start):
    tour = [start, node terdekat dari start]
    unvisited = semua node kecuali start dan node terdekat
    while unvisited tidak kosong:
        v = node di unvisited yang paling dekat dengan sub-tour
        posisi = posisi penyisipan v yang menghasilkan Δ terkecil
        tour.insert(posisi, v)
        unvisited.remove(v)
    return tour
```

3.5.3 Farthest Neighbor

Pseudocode

```
function farthestNeighbor(nodes, start):
    tour = [start]
    unvisited = semua node kecuali start
    current = start
    while unvisited tidak kosong:
        next = node di unvisited dengan jarak terjauh dari current
        tour.append(next)
        unvisited.remove(next)
        current = next
    return tour
```

3.5.4 Arbitrary Solution

Pseudocode

```
function arbitrarySolution(nodes, seed):
    random = PRNG(seed)
    tour = salinan nodes
    shuffle(tour, random)
    return tour
```

3.5.5 2-opt

Pseudocode

```
function twoOpt(tour, maxPass):
    pass = 0
    improved = true
```

```
    while improved dan pass < maxPass:
        improved = false
        for i dari 1 sampai n-1:
            for j dari i+1 sampai n:
                Δ = hitung perubahan biaya 2-opt
                if Δ < 0:
                    balik segmen tour[i..j]
                    improved = true
        pass = pass + 1
    return tour
```

3.5.6 Tabu Search

Pseudocode

```
function tabuSearch(tour, maxIter, tabuTenure):
    best = salinan tour
    tabuList = []
    for iter dari 1 sampai maxIter:
        kandidat terbaik = null
        for setiap kandidat 2-opt:
            if kandidat tidak tabu atau memenuhi aspiration criteria:
                if kandidat terbaik null atau biaya kandidat < biaya kandidat terbaik:
                    kandidat terbaik = kandidat
        if kandidat terbaik tidak null:
            terapkan kandidat terbaik
            tambahkan pasangan titik ke tabuList
            if panjang tabuList > tabuTenure:
                hapus elemen tertua
            if biaya tour < biaya best:
                best = salinan tour
    return best
```

3.5.7 Simulated Annealing

Pseudocode

```
function simulatedAnnealing(tour, T0, Tmin, alpha, iterPerT):
    T = T0
    best = salinan tour
    while T > Tmin:
        for i dari 1 sampai iterPerT:
            kandidat = pilih 2-opt secara acak
            Δ = biaya kandidat - biaya tour
            if Δ < 0 atau random() < exp(-Δ/T):
                terapkan kandidat
                if biaya tour < biaya best:
                    best = salinan tour
```

```
    T = T * alpha
  return best
```

3.6 Perancangan Antarmuka Pengguna

Antarmuka sistem terdiri dari beberapa bagian utama:

1. Panel data:
    - Memilih sumber data (data bawaan, file .xlsx, atau data acak).
    - Mengelola titik data (menambah, menghapus, mengedit).
    - Menampilkan daftar titik data.
2. Panel metode:
    - Memilih metode konstruksi.
    - Memilih titik awal.
    - Memilih metode perbaikan.
    - Mengatur parameter algoritma.
3. Area Utama:
    - Menampilkan titik dan rute yang dihasilkan pada bidang dua dimensi.
    - Menampilkan grafik perkembangan biaya.
    - Menampilkan pseudocode yang tersinkronisasi dengan langkah algoritma.
4. Kontrol Visualisasi:
    - Tombol kembali ke langkah sebelumnya.
    - Tombol pause/langkah.
    - Opsi menampilkan label
    - Opsi menampilkan rute terbaik
5. Panel Hasil
    - Menampilkan metode konstruksi dan perbaikan.
    - Menampilkan biaya solusi awal dan akhir.
    - Menampilkan persentase perbaikan.
    - Menampilkan waktu komputasi.
    - Menampilkan urutan rute akhir.
    - Menampilkan tabel perbandingan beberapa eksekusi

3.7 Metrik Evaluasi

Metrik evaluasi yang digunakan dalam eksperimen adalah:

1. Biaya Awal: Total jarak rute yang dihasilkan oleh metode konstruksi sebelum perbaikan.
2. Biaya Akhir: Total jarak rute setelah proses perbaikan.
3. Persentase Perbaikan: Mengukur seberapa besar penurunan biaya dari solusi awal ke solusi akhir.
4. Waktu Komputasi: Waktu yang dibutuhkan untuk menjalankan algoritma, diukur dalam milidetik.

3.8 Pengujian Sistem

Pengujian sistem dilakukan terhadap fungsi algoritma, validasi parameter, dan pembacaan data. Untuk memastikan bahwa sistem berjalan sesuai dengan yang direncanakan harus diuji dengan beberapa skenario:

No.	Skenario Pengujian	Hasil yang diharapkan	Hasil
1	Nearest Neighbor pada lima titik data kota	Rute dan biaya sama dengan hasil yang di notebook	Lulus
2	Tabu Search pada lima titik data kota	Biaya sama dengan hasil notebook	Lulus
3	Dua belas kombinasi metode konstruksi dan perbaikan pada 25 titik data kota	Rute merupakan sebuah permutasi yang valid dan tidak ada biaya akhir yang lebih buruk	Lulus
4	Kondisi berhenti 2-opt	2-opt berhenti ketika tidak ada kenaikan biaya kurang dari 0	Lulus
5	Reprodusibilitas Simulated Annealing	Seed yang sama menghasilkan rute dan jumlah langkah sama	Lulus
6	Sinkronisasi *pseudocode*	Pseudocode berjalan sesuai dengan tampilan hasil	Lulus
7	Validasi Parameter	Parameter di luar rentang ditolak	Lulus
8	Parser data teks	Dapat membaca data tanpa header dan melompati baris yang tidak sesuai	Lulus
9	Parser koordinat geografis	Kolom lat/lng dikenali dan data kosong dilaporkan	Lulus

Tabel 3.1 Skenario Pengujian Sistem

Berdasarkan hasil di atas, dapat disimpulkan bahwa sistem berjalan sesuai dengan rancangan sistem sehingga mengurangi risiko terjadinya kegagalan sistem.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Rancangan Eksperimen

Eksperimen dibagi menjadi dua bagian, yaitu pengujian kombinasi metode antara konstruksi dan perbaikan, dan pengujian perubahan parameter metaheuristik. Pembagian ini dilakukan untuk mendapatkan kemampuan setiap kombinasi metode, kemudian melihat pengaruh parameter Tabu Search dan Simulated Annealing secara terpisah.

4.1.1 Pengujian kombinasi metode

Pengujian utama menggunakan empat ukuran dataset:

1. Dataset bawaan (5 kota)
2. Dataset acak dengan 25 titik
3. Dataset acak dengan 50 titik
4. Dataset acak dengan 100 titik

Dataset yang sama digunakan untuk membandingkan setiap kombinasi. Hasil analisis menunjukan biaya akhir, persentase perbaikan, dan waktu komputasinya.

Empat metode konstruksi yang digunakan:

- Nearest Neighbor
- Nearest Insertion
- Farthest Neighbor
- Arbitrary

Solusi awal dibangun kemudian dilanjutkan dengan metode perbaikan:

- Tanpa perbaikan
- 2-opt
- Tabu Search
- Simulated Annealing

Dengan demikian, terdapat 16 kombinasi metode dan untuk ukuran 25, 50 , dan 100 titik menggunakan 5 dataset masing-masing sehingga total 256 pengujian yang dilakukan. Berikut parameter yang digunakan dalam proses eksperimen:

Parameter	Nilai
Maksimum *pass* 2-opt	100
Maksimum iterasi Tabu Search	100
Tabu *tenure*	10
Temperatur awal Simulated Annealing	1000
Temperatur minimum	0,001
Cooling rate	0,995
Iterasi per temperatur	100

Tabel 4.1 Parameter Tetap Eksperimen Pertama

4.1.2 Pengujian perubahan parameter metaheuristik

Eksperimen ini dilakukan untuk mengetahui pengaruh parameter terhadap hasil Tabu Search dan Simulated Annealing. Pengujian dilakukan pada dataset dengan 100 kota dan Nearest Neighbor sebagai konstruktif utamanya, titik awal dimulai dari kota atau titik 001.

Biaya awal yang dihasilkan oleh Nearest Neighbor sebesar 979,96. Pengujian dilakukan dengan cara mengubah 1 parameter untuk setiap kelompok pengujian. Variasi parameter yang digunakan sebagai berikut:

Metode	Parameter yang diubah	Nilai yang diuji	Parameter tetap
Tabu Search	Maksimum iterasi	25, 50, 100, 200, 500	Tabu tenure = 10
Tabu Search	Tabu tenure	3, 5, 10, 20, 30	Maksimum iterasi = 100
Simulated Annealing	Temperatur awal	100, 500, 1.000, 2.000, 5.000	Temperatur minimum = 0,001; cooling rate = 0,995; iterasi per temperatur = 100
Simulated Annealing	Cooling rate	0,900; 0,950; 0,980; 0,995; 0,999	Temperatur awal = 1.000; temperatur minimum = 0,001; iterasi per temperatur = 100
Simulated Annealing	Iterasi per temperatur	25, 50, 100, 200, 500	Temperatur awal = 1.000; temperatur minimum = 0,001; cooling rate = 0,995

Tabel 4.2 Parameter Uji Eksperimen Kedua

Setiap kelompok parameter memiliki 5 nilai parameter yang diubah sehingga total eksperimen yang dijalankan sebanyak 25 run. Total run yang dijalankan pada studi ini sebanyak 281 run.

Analisis perubahan parameter didasarkan pada dua ukuran utama, yaitu biaya akhir dan waktu komputasinya. Biaya akhir untuk menilai kualitas rute atau solusi yang dibuat dan waktu komputasi digunakan untuk mempertimbangkan biaya tambahan akibat perubahan parameter tersebut. Setiap kombinasi dijalankan untuk menguji sensitivitas parameter terhadap dataset.

4.2 Hasil pada Data Lima Kota

Data bawaan terdiri dari Jakarta (0,0), Bandung (2,3), Yogyakarta (5,1), Surabaya (6,5), dan Malang (8,2). Berikut merupakan masing-masing biaya yang dihasilkan dari setiap metode konstruksi diikuti dengan setiap metode perbaikan:

Metode Konstruksi	Tanpa Perbaikan	2-opt	Tabu Search	Simulated Annealing
Nearest Neighbor	21,7892	19,9445	19,9445	19,9445
Nearest Insertion	19,9445	19,9445	19,9445	19,9445
Farthest Neighbor	28,0232	19,9445	19,9445	19,9445
Arbitrary	27,2964	19,9445	19,9445	19,9445

Tabel 4.3 Hasil Metode Konstruksi dan Metode Perbaikan pada 5 Titik

Berdasarkan hasil tabel di atas, dapat dilihat bahwa Nearest Insertion menghasilkan biaya terbaik dengan tanpa perbaikan, diikuti dengan seluruh metode perbaikan yang membawa seluruh solusi awal ke biaya yang sama.

4.3 Perbandingan Solusi Awal

Jumlah Titik	Nearest Neighbor	Nearest Insertion	Farthest Neighbor	Arbitrary
25	513,45	490,56	1777,60	1333,99
50	707,33	691,46	3783,84	2746,93
100	999,63	957,81	7561,43	5212,46

Tabel 4.4 Perbandingan Solusi Awal Metode Perbaikan

Nearest Insertion menghasilkan solusi awal yang terbaik pada ketiga ukuran dataset. Sedangkan untuk Nearest Neighbor berada pada peringkat kedua dan menghasilkan solusi awal yang cukup baik dengan algoritma yang lebih sederhana. Farthest Neighbor menghasilkan solusi awal yang paling tinggi biayanya begitu pula biaya tinggi dimana diambil rute secara acak tanpa memperhatikan kedekatan jaraknya.

4.4 Perbandingan Metode Perbaikan

Jumlah Titik	Metode	Rata-rata biaya awal	Rata-rata biaya akhir	Rata-rata perbaikan	Rata-rata waktu
25	2-opt	1028,9	430,12	42,92%	0,17 ms
25	Tabu Search	1028,9	422,15	43,77%	1,8 ms

25	Simulated Annealing	1028,9	418,09	44,29%	34,95 ms
50	2-opt	1982,39	614,87	46,84%	0,45 ms
50	Tabu Search	1982,39	601,07	47,54%	5,24 ms
50	Simulated Annealing	1982,39	601,15	47,52%	47,75 ms
100	2-opt	3682,83	858,41	49,38%	2,1 ms
100	Tabu Search	3682,83	850,79	49,87%	22,49 ms
100	Simulated Annealing	3682,83	828,46	50,91%	77,6 ms

Tabel 4.5 Perbandingan Solusi Metode Perbaikan

Seluruh metode perbaikan mampu menurunkan biaya secara signifikan. Berdasarkan hasil solusi awal sebelumnya, metode perbaikan juga mampu menurunkan biaya tinggi dari solusi awal Farthest Neighbor dan Arbitrary dengan signifikan, ini menunjukan bahwa metode yang digunakan efektif untuk permasalahan.

Berdasarkan waktu komputasi, 2-opt memiliki performa yang paling baik dengan penurunan biaya yang besar. Kurangnya dibandingkan metode *metaheuristic* seperti Tabu Search dan Simulated Annealing, yaitu solusi berhenti pada lokal optimum.

Tabu Search menghasilkan biaya lebih rendah dibandingkan metode 2-opt pada semua ukuran data, walaupun perbedaan waktu yang cukup signifikan (10-12 kali lebih tinggi). Begitu pula, untuk Simulated Annealing dimana menghasilkan biaya terbaik pada 25 dan 100 titik. Pada dataset dengan titik 100, menghasilkan solusi dengan biaya rata-rata sebesar 828,46, lebih rendah 29,95 dibandingkan 2-opt dan 22,3 dibandingkan Tabu Search. Namun, biaya komputasi untuk Simulated Annealing cukup besar karena banyaknya tingkat temperatur sekitar 2757 dengan iterasi sebanyak 275700.

4.5 Pengaruh Solusi Awal

Eksperimen menunjukan bahwa kualitas metode konstruksi memiliki pengaruh yang besar terhadap biaya saat sebelum terjadi perbaikan, dengan Nearest Insertion menjadi pilihan terbaik untuk solusi awal.

Walaupun solusi awal buruk, eksperimen juga menunjukan bahwa dengan adanya metode perbaikan dari 2-opt maupun *metaheuristic* mampu membawakan solusi awal yang buruk hingga mencapai solusi akhir yang baik. Oleh karena itu, berdasarkan hasil di atas, gabungan Nearest Insertion dapat digunakan untuk memperoleh solusi awal yang rendah. Tabu Search atau Simulated Annealing dapat digunakan ketika penurunan biaya lebih diutamakan daripada waktu komputasi.

4.6 Pengaruh Perubahan Parameter Metaheuristik

Eksperimen tambahan dilakukan untuk menganalisis pengaruh parameter Tabu Search dan Simulated Annealing dengan dataset yang terdiri dari 100 titik. Nearest Neighbor sebagai metode konstruksi dengan titik 001 sebagai titik awal. Nearest Neighbor menghasilkan sebesar 979,96 untuk biaya awalnya. Dalam kelompok pengujian, hanya mengubah satu parameter sedangkan parameter lainnya menggunakan *default.*

4.6.1 Pengaruh Maksimum Iterasi Tabu Search

Pengujian dengan tabu tenure sebesar 10

Maksimum iterasi	Biaya akhir	Perbaikan	Waktu
25	812,66	17,1%	20 ms
50	810,34	17,3%	34 ms
100	810,34	17,3%	43 ms
200	810,34	17,3%	74 ms
500	810,34	17,3%	159 ms

Tabel 4.6 Perbandingan Solusi Perubahan Maksimum Iterasi Tabu Search

Peningkatan jumlah iterasi dari 25 hingga 50 menurunkan biaya dari 812,66 menjadi 810,34. Penambahan iterasi dari 50 hingga 500 tidak menghasilkan solusi yang lebih baik, hanya menambah waktu komputasi dari 34 ms menjadi 159 ms.

4.6.2 Pengaruh Tabu Tenure
Pengujian tabu tenure menggunakan maksimum iterasi sebesar 100.

Tabu tenure	Biaya akhir	Perbaikan	Waktu
3	810,34	17,3%	36 ms
5	810,34	17,3%	27 ms
10	810,34	17,3%	34 ms
20	803,29	18,0%	58 ms
30	796,91	18,7%	64 ms

Tabel 4.7 Perbandingan Solusi Perubahan Maksimum Iterasi Tabu Search

Tabu tenure 3,5, dan 10 menghasilkan biaya akhir yang sama, yaitu 810,34.. Kemudian ketika tenure dinaikkan ke 20, biaya turun menjadi 803,29. Dengan biaya terendah pada tenure 30, yaitu 796,91.

Tenure yang lebih panjang mencegah algoritma untuk menggunakan pertukaran yang baru dilakukan dengan jumlah iterasi yang lebih banyak. Untuk dataset ini, tenure 20 dan 30 membantu mencari solusi baru yang lebih baik dibandingkan tenure yang lebih pendek. Namun, waktu komputasi meningkat menjadi 58 ms dan 64 ms.

4.6.3 Pengaruh Temperatur Awal Simulated Annealing

Pengujian temperatur awal menggunakan parameter tetap untuk temperatur minimum sebesar 0,001, cooling rate 0,995, dan 100 iterasi per temperatur.

Temperatur awal	Biaya akhir	Perbaikan	Waktu
100	797,91	18,6%	169 ms
500	807,97	17,6%	342 ms
1.000	810,22	17,3%	451 ms
2.000	835,39	14,8%	516 ms
5.000	797,39	18,6%	582 ms

Tabel 4.8 Perbandingan Solusi Perubahan Temperatur Awal Simulated Annealing

Perubahan temperatur awal tidak menghasilkan pola penurunan biaya yang berurutan. Temperatur awal 5000 menghasilkan terendah sebesar 797,39, tetapi membutuhkan waktu komputasi sebesar 582 ms. Sedangkan pada temperatur awal 100 menghasilkan biaya sebesar 797,91 dalam waktu 169 ms.

Selisih biaya antara 100 dan 5000 hanya 0,52, sedangkan temperatur 5000 membutuhkan waktu sekitar 3,4 kali lipat lebih lama dibandingkan temperatur 100. Pada run ini, temperatur awal yang lebih tinggi tidak selalu menghasilkan biaya yang lebih rendah.

4.6.4 Pengaruh Cooling Rate Simulated Annealing

Pengujian cooling rate menggunakan parameter tetap yaitu parameter awal 1000, temperatur minimum 0,001, dan 100 iterasi per temperatur.

Cooling rate	Biaya akhir	Perbaikan	Waktu
0,900	936,86	4,4%	25 ms
0,950	875,73	10,6%	63 ms
0,980	891,90	9,0%	111 ms
0,995	810,22	17,3%	417 ms

0,999	791,66	19,2%	2.018 ms

Tabel 4.9 Perbandingan Solusi Perubahan Cooling Rate Simulated Annealing

Cooling rate 0,900 menghasilkan proses pendinginan yang lebih cepat dengan waktu komputasi sebesar 25 ms. Namun biaya akhir masih relatif lebih besar pada 936,86. Cooling rate 0,999 menghasilkan biaya terendah sebesar 791,66, karena temperatur yang turun lebih lambat sehingga waktu pencarian lebih lama pada 2018 ms.

Peningkatan kualitas solusi disertai kenaikan waktu komputasi, dengan cooling rate 0,999 membutuhkan waktu yang lebih lama hampir lima kali lipat dibandingkan cooling rate 0,995. Hasil cooling rate 0,98 juga lebih tinggi dibandingkan cooling rate 0,95, ini menunjukan bahwa tidak selalu monoton antara hubungan parameter dan hasil.

4.6.5 Pengaruh Iterasi per Temperatur

Pengujian iterasi pada Simulated Annealing menggunakan parameter temperatur awal 1000, temperatur minimum 0,001, dan cooling rate 0,995.

Iterasi per temperatur	Biaya akhir	Perbaikan	Waktu
25	847,05	13,6%	121 ms
50	821,57	16,2%	240 ms
100	810,22	17,3%	472 ms
200	802,58	18,1%	887 ms
500	789,44	19,4%	2.164 ms

Tabel 4.10 Perbandingan Solusi Perubahan Iterasi per Temperatur Simulated Annealing

Pada uji ini, dapat dilihat bahwa seiring bertambahnya jumlah iterasinya, maka terjadi penurunan biaya akhir. Biaya turun dari 847,05 pada iterasi 25 menjadi 789,44 pada 500 iterasi. Namun, ini diikuti juga dengan waktu komputasi yang lebih besar dari 121 ms menjadi 2164 ms. Konfigurasi dengan 500 iterasi menghasilkan biaya terendah dibandingkan konfigurasi lainnya, tetapi membutuhkan waktu komputasi yang lebih tinggi.

4.6.6 Perbandingan Hasil Parameter

Hasil terbaik Tabu Search diperoleh menggunakan maksimum 100 iterasi dan tabu tenure 30, dengan biaya akhir 796,91 dan waktu 64 ms. Hasil terbaik Simulated Annealing diperoleh dengan parameter 500 iterasi per temperatur, dengan biaya akhir 789,44 dan waktu komputasi sebesar 2164 ms.

Simulated Annealing menghasilkan biaya 7,47 lebih rendah dibandingkan Tabu Search, tetapi membutuhkan waktu sekitar 33,8 kali lipat lebih lama. Oleh karena itu, pemilihan parameter tidak dapat didasari hanya berdasarkan biaya akhir saja, melainkan juga waktu yang diperlukan untuk mencari solusi harus dipertimbangkan juga.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Sistem berbasis web mampu berhasil mengintegrasikan pemasukan data, pembentukan solusi awal, proses perbaikan, visualisasi langkah, dan perbandingan hasil simulasi TSP antar metode pada satu *interface*.
2. Nearest Insertion menghasilkan rata-rata solusi awal terbaik pada data 25,50, dan 100 titik. Pada data lima kota, metode menghasilkan biaya terendah sebesar 19,9445.
3. 2-opt memiliki waktu komputasi paling rendah dibandingkan Tabu Search dan Simulated Annealing. Namun Tabu Search dapat menghasilkan solusi dengan biaya lebih rendah pada seluruh ukuran data, sedangkan pada Simulated Annealing menghasilkan hasil terbaik pada data ukuran 25 dan 100 titik, dengan waktu komputasi yang lebih besar.
4. Perubahan parameter mempengaruhi kualitas solusi dan waktu komputasi. Hasil terbaik terdapat pada Tabu Search dengan biaya 796,91 menggunakan tabu tenure 30, sedangkan pada Simulated Annealing dengan biaya 789,44 menggunakan 500 iterasi per temperatur.
5. Tidak ada metode yang lebih unggul dari sisi kualitas solusi dan waktu secara bersamaan. Semua harus dipertimbangkan berdasarkan ketersediaan sumber daya dengan mempertimbangkan waktu komputasi dan kualitas solusi.

5.2 Saran

1. Setiap konfigurasi harus diuji dengan beberapa seed untuk stabilitas hasil.
2. Tabel perbandingan bisa ditambah dengan nilai parameter agar lebih mudah dicatat.
3. Pengembangan lebih lanjut bisa menambahkan metode konstruksi lainnya seperti Cheapest Insertion, lalu menguji kombinasi dengan metode metaheuristik untuk dataset yang lebih beragam.

DAFTAR PUSTAKA

Croes, G. A. (1958). A method for solving traveling-salesman problems. Operations Research, 6(6).

Glover, F. (1989). Tabu search—Part I. ORSA Journal on Computing, 1(3).

Johnson, D. S., & McGeoch, L. A. (1997). The traveling salesman problem: A case study in local optimization. Dalam E. H. L. Aarts & J. K. Lenstra (Ed.). John Wiley & Sons.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598).

Talbi, E.-G. (2009). Metaheuristics: From design to implementation. John Wiley & Sons.

LAMPIRAN

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	**Nearest Neighbor**	**Tabu Search**	**25**	**548.25**	**406.16**	**25.9**	**6**
2	Nearest Neighbor	Tanpa improvement	25	548.25	548.25	0.0	0
3	Nearest Neighbor	2-opt	25	548.25	406.16	25.9	1
4	**Nearest Neighbor**	**Simulated Annealing**	**25**	**548.25**	**406.16**	**25.9**	**80**
5	Nearest Insertion	Tanpa improvement	25	503.59	503.59	0.0	1
6	Nearest Insertion	2-opt	25	503.59	421.36	16.3	0
7	**Nearest Insertion**	**Tabu Search**	**25**	**503.59**	**406.16**	**19.3**	**4**
8	**Nearest Insertion**	**Simulated Annealing**	**25**	**503.59**	**406.16**	**19.3**	**70**
9	Farthest Neighbor	Tanpa improvement	25	1790.35	1790.35	0.0	0
10	Farthest Neighbor	2-opt	25	1790.35	420.44	76.5	1
11	Farthest Neighbor	Tabu Search	25	1790.35	420.44	76.5	3
12	Farthest Neighbor	Simulated Annealing	25	1790.35	406.16	77.3	55
13	Arbitrary (acak)	Tanpa improvement	25	1279.67	1279.67	0.0	0
14	Arbitrary (acak)	2-opt	25	1279.67	422.31	67.0	0
15	Arbitrary (acak)	Tabu Search	25	1279.67	422.31	67.0	3
16	Arbitrary (acak)	Simulated Annealing	25	1279.67	406.16	68.3	86

Lampiran 1. Tampilan Perbandingan Hasil Run pada 25 Titik Seed 1

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	25	556.41	556.41	0.0	1
2	Nearest Neighbor	2-opt	25	556.41	499.18	10.3	1
3	Nearest Neighbor	Tabu Search	25	556.41	499.18	10.3	4
4	Nearest Neighbor	Simulated Annealing	25	556.41	484.18	13.0	53
5	Nearest Insertion	Tanpa improvement	25	513.46	513.46	0.0	1
6	Nearest Insertion	2-opt	25	513.46	501.35	2.4	0
7	Nearest Insertion	Tabu Search	25	513.46	501.24	2.4	3
8	Nearest Insertion	Simulated Annealing	25	513.46	487.48	5.1	79
9	Farthest Neighbor	Tanpa improvement	25	1937.67	1937.67	0.0	0
10	Farthest Neighbor	2-opt	25	1937.67	487.59	74.8	1
11	Farthest Neighbor	Tabu Search	25	1937.67	487.48	74.8	4
12	**Farthest Neighbor**	**Simulated Annealing**	**25**	**1937.67**	**483.35**	**75.1**	**68**
13	Arbitrary (acak)	Tanpa improvement	25	1480.33	1480.33	0.0	0
14	Arbitrary (acak)	2-opt	25	1480.33	489.28	66.9	0
15	Arbitrary (acak)	Tabu Search	25	1480.33	487.48	67.1	3
16	Arbitrary (acak)	Simulated Annealing	25	1480.33	484.18	67.3	57

Lampiran 2. Tampilan Perbandingan Hasil Run pada 25 Titik Seed 2

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	25	397.78	397.78	0.0	1
2	Nearest Neighbor	2-opt	25	397.78	397.78	0.0	1
3	Nearest Neighbor	Tabu Search	25	397.78	395.06	0.7	5
4	Nearest Neighbor	Simulated Annealing	25	397.78	390.94	1.7	55
5	Nearest Insertion	Tanpa improvement	25	421.37	421.37	0.0	1
6	Nearest Insertion	2-opt	25	421.37	403.30	4.3	0
7	Nearest Insertion	Tabu Search	25	421.37	395.78	6.1	4
8	Nearest Insertion	Simulated Annealing	25	421.37	390.57	7.3	57
9	Farthest Neighbor	Tanpa improvement	25	1640.51	1640.51	0.0	0
10	Farthest Neighbor	2-opt	25	1640.51	422.87	74.2	1
11	Farthest Neighbor	Tabu Search	25	1640.51	390.94	76.2	4
12	Farthest Neighbor	Simulated Annealing	25	1640.51	390.57	76.2	87
13	Arbitrary (acak)	Tanpa improvement	25	1278.60	1278.60	0.0	0
14	Arbitrary (acak)	2-opt	25	1278.60	423.43	66.9	0
15	**Arbitrary (acak)**	**Tabu Search**	**25**	**1278.60**	**390.57**	**69.5**	**3**
16	**Arbitrary (acak)**	**Simulated Annealing**	**25**	**1278.60**	**390.57**	**69.5**	**60**

Lampiran 3. Tampilan Perbandingan Hasil Run pada 25 Titik Seed 3

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	25	591.26	591.26	0.0	0
2	Nearest Neighbor	2-opt	25	591.26	439.60	25.7	1
3	Nearest Neighbor	Tabu Search	25	591.26	439.60	25.7	3
4	Nearest Neighbor	Simulated Annealing	25	591.26	439.60	25.7	76
5	Nearest Insertion	Tanpa improvement	25	521.55	521.55	0.0	0
6	Nearest Insertion	2-opt	25	521.55	449.73	13.8	0
7	Nearest Insertion	Tabu Search	25	521.55	439.60	15.7	4
8	Nearest Insertion	Simulated Annealing	25	521.55	439.60	15.7	74
9	Farthest Neighbor	Tanpa improvement	25	1672.86	1672.86	0.0	0
10	Farthest Neighbor	2-opt	25	1672.86	439.61	73.7	0
11	Farthest Neighbor	Tabu Search	25	1672.86	439.60	73.7	3
12	**Farthest Neighbor**	**Simulated Annealing**	**25**	**1672.86**	**437.09**	**73.9**	**60**
13	Arbitrary (acak)	Tanpa improvement	25	1250.23	1250.23	0.0	0
14	Arbitrary (acak)	2-opt	25	1250.23	439.60	64.8	0
15	Arbitrary (acak)	Tabu Search	25	1250.23	439.60	64.8	3
16	**Arbitrary (acak)**	**Simulated Annealing**	**25**	**1250.23**	**437.09**	**65.0**	**50**

Lampiran 4. Tampilan Perbandingan Hasil Run pada 25 Titik Seed 4

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	25	473.55	473.55	0.0	0
2	**Nearest Neighbor**	**2-opt**	**25**	**473.55**	**370.48**	**21.8**	**1**
3	**Nearest Neighbor**	**Tabu Search**	**25**	**473.55**	**370.48**	**21.8**	**3**
4	Nearest Neighbor	Simulated Annealing	25	473.55	370.48	21.8	70
5	Nearest Insertion	Tanpa improvement	25	492.85	492.85	0.0	0
6	Nearest Insertion	2-opt	25	492.85	381.35	22.6	0
7	**Nearest Insertion**	**Tabu Search**	**25**	**492.85**	**370.48**	**24.8**	**4**
8	**Nearest Insertion**	**Simulated Annealing**	**25**	**492.85**	**370.48**	**24.8**	**63**
9	Farthest Neighbor	Tanpa improvement	25	1846.61	1846.61	0.0	0
10	Farthest Neighbor	2-opt	25	1846.61	405.56	78.0	1
11	**Farthest Neighbor**	**Tabu Search**	**25**	**1846.61**	**370.48**	**79.9**	**3**
12	**Farthest Neighbor**	**Simulated Annealing**	**25**	**1846.61**	**370.48**	**79.9**	**75**
13	Arbitrary (acak)	Tanpa improvement	25	1381.14	1381.14	0.0	0
14	Arbitrary (acak)	2-opt	25	1381.14	381.35	72.4	0
15	**Arbitrary (acak)**	**Tabu Search**	**25**	**1381.14**	**370.48**	**73.2**	**3**
16	**Arbitrary (acak)**	**Simulated Annealing**	**25**	**1381.14**	**370.48**	**73.2**	**52**

Lampiran 5. Tampilan Perbandingan Hasil Run pada 25 Titik Seed 5

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	50	720.13	720.13	0.0	1
2	Nearest Neighbor	2-opt	50	720.13	579.75	19.5	3
3	**Nearest Neighbor**	**Tabu Search**	**50**	**720.13**	**569.97**	**20.9**	**12**
4	Nearest Neighbor	Simulated Annealing	50	720.13	597.30	17.1	71
5	Nearest Insertion	Tanpa improvement	50	671.90	671.90	0.0	1
6	Nearest Insertion	2-opt	50	671.90	590.92	12.1	2
7	Nearest Insertion	Tabu Search	50	671.90	589.00	12.3	10
8	Nearest Insertion	Simulated Annealing	50	671.90	572.88	14.7	64
9	Farthest Neighbor	Tanpa improvement	50	3841.43	3841.43	0.0	0
10	Farthest Neighbor	2-opt	50	3841.43	579.75	84.9	2
11	**Farthest Neighbor**	**Tabu Search**	**50**	**3841.43**	**569.97**	**85.2**	**12**
12	Farthest Neighbor	Simulated Annealing	50	3841.43	591.87	84.6	65
13	Arbitrary (acak)	Tanpa improvement	50	2569.25	2569.25	0.0	0
14	Arbitrary (acak)	2-opt	50	2569.25	674.29	73.8	3
15	Arbitrary (acak)	Tabu Search	50	2569.25	598.53	76.7	10
16	Arbitrary (acak)	Simulated Annealing	50	2569.25	584.86	77.2	63

Lampiran 6. Tampilan Perbandingan Hasil Run pada 50 Titik Seed 1

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	50	788.15	788.15	0.0	1
2	Nearest Neighbor	2-opt	50	788.15	620.58	21.3	2
3	**Nearest Neighbor**	**Tabu Search**	**50**	**788.15**	**607.56**	**22.9**	**11**
4	Nearest Neighbor	Simulated Annealing	50	788.15	622.49	21.0	74
5	Nearest Insertion	Tanpa improvement	50	715.27	715.27	0.0	1
6	Nearest Insertion	2-opt	50	715.27	626.05	12.5	2
7	Nearest Insertion	Tabu Search	50	715.27	619.45	13.4	11
8	Nearest Insertion	Simulated Annealing	50	715.27	632.57	11.6	85
9	Farthest Neighbor	Tanpa improvement	50	3980.23	3980.23	0.0	0
10	Farthest Neighbor	2-opt	50	3980.23	626.55	84.3	2
11	Farthest Neighbor	Tabu Search	50	3980.23	616.59	84.5	10
12	Farthest Neighbor	Simulated Annealing	50	3980.23	622.49	84.4	80
13	Arbitrary (acak)	Tanpa improvement	50	3030.46	3030.46	0.0	0
14	Arbitrary (acak)	2-opt	50	3030.46	618.24	79.6	2
15	Arbitrary (acak)	Tabu Search	50	3030.46	618.24	79.6	10
16	Arbitrary (acak)	Simulated Annealing	50	3030.46	631.93	79.1	54

Lampiran 7. Tampilan Perbandingan Hasil Run pada 50 Titik Seed 2

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	50	634.95	634.95	0.0	1
2	Nearest Neighbor	2-opt	50	634.95	551.92	13.1	2
3	**Nearest Neighbor**	**Tabu Search**	**50**	**634.95**	**551.92**	**13.1**	**10**
4	Nearest Neighbor	Simulated Annealing	50	634.95	563.74	11.2	141
5	Nearest Insertion	Tanpa improvement	50	652.04	652.04	0.0	1
6	Nearest Insertion	2-opt	50	652.04	572.17	12.2	2
7	Nearest Insertion	Tabu Search	50	652.04	565.35	13.3	8
8	Nearest Insertion	Simulated Annealing	50	652.04	566.08	13.2	88
9	Farthest Neighbor	Tanpa improvement	50	3454.09	3454.09	0.0	0
10	Farthest Neighbor	2-opt	50	3454.09	569.43	83.5	1
11	Farthest Neighbor	Tabu Search	50	3454.09	565.35	83.6	12
12	Farthest Neighbor	Simulated Annealing	50	3454.09	564.64	83.7	67
13	Arbitrary (acak)	Tanpa improvement	50	2609.52	2609.52	0.0	0
14	Arbitrary (acak)	2-opt	50	2609.52	591.50	77.3	2
15	Arbitrary (acak)	Tabu Search	50	2609.52	578.38	77.8	9
16	Arbitrary (acak)	Simulated Annealing	50	2609.52	555.91	78.7	55

Lampiran 8. Tampilan Perbandingan Hasil Run pada 50 Titik Seed 3

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	50	683.73	683.73	0.0	1
2	Nearest Neighbor	2-opt	50	683.73	625.25	8.6	6
3	Nearest Neighbor	Tabu Search	50	683.73	625.25	8.6	16
4	**Nearest Neighbor**	**Simulated Annealing**	**50**	**683.73**	**610.30**	**10.7**	**56**
5	Nearest Insertion	Tanpa improvement	50	765.13	765.13	0.0	2
6	Nearest Insertion	2-opt	50	765.13	630.88	17.5	5
7	Nearest Insertion	Tabu Search	50	765.13	630.88	17.5	17
8	Nearest Insertion	Simulated Annealing	50	765.13	615.13	19.6	46
9	Farthest Neighbor	Tanpa improvement	50	3738.69	3738.69	0.0	1
10	Farthest Neighbor	2-opt	50	3738.69	661.75	82.3	2
11	Farthest Neighbor	Tabu Search	50	3738.69	627.30	83.2	17
12	Farthest Neighbor	Simulated Annealing	50	3738.69	614.07	83.6	42
13	Arbitrary (acak)	Tanpa improvement	50	2722.71	2722.71	0.0	1
14	Arbitrary (acak)	2-opt	50	2722.71	672.59	75.3	2
15	Arbitrary (acak)	Tabu Search	50	2722.71	643.52	76.4	27
16	Arbitrary (acak)	Simulated Annealing	50	2722.71	627.22	77.0	41

Lampiran 9. Tampilan Perbandingan Hasil Run pada 50 Titik Seed 4

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	50	709.69	709.69	0.0	1
2	Nearest Neighbor	2-opt	50	709.69	605.60	14.7	4
3	Nearest Neighbor	Tabu Search	50	709.69	605.60	14.7	19
4	**Nearest Neighbor**	**Simulated Annealing**	**50**	**709.69**	**590.31**	**16.8**	**49**
5	Nearest Insertion	Tanpa improvement	50	652.94	652.94	0.0	2
6	Nearest Insertion	2-opt	50	652.94	632.09	3.2	1
7	Nearest Insertion	Tabu Search	50	652.94	623.80	4.5	15
8	Nearest Insertion	Simulated Annealing	50	652.94	591.22	9.5	42
9	Farthest Neighbor	Tanpa improvement	50	3904.74	3904.74	0.0	1
10	Farthest Neighbor	2-opt	50	3904.74	639.13	83.6	3
11	Farthest Neighbor	Tabu Search	50	3904.74	602.49	84.6	18
12	Farthest Neighbor	Simulated Annealing	50	3904.74	598.20	84.7	50
13	Arbitrary (acak)	Tanpa improvement	50	2802.70	2802.70	0.0	0
14	Arbitrary (acak)	2-opt	50	2802.70	629.03	77.6	2
15	Arbitrary (acak)	Tabu Search	50	2802.70	612.21	78.2	20
16	Arbitrary (acak)	Simulated Annealing	50	2802.70	594.66	78.8	60

Lampiran 10. Tampilan Perbandingan Hasil Run pada 50 Titik Seed 5

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	100	979.96	979.96	0.0	2
2	Nearest Neighbor	2-opt	100	979.96	810.34	17.3	12
3	Nearest Neighbor	Tabu Search	100	979.96	810.34	17.3	39
4	Nearest Neighbor	Simulated Annealing	100	979.96	842.72	14.0	60
5	Nearest Insertion	Tanpa improvement	100	953.06	953.06	0.0	4
6	Nearest Insertion	2-opt	100	953.06	854.65	10.3	7
7	Nearest Insertion	Tabu Search	100	953.06	841.23	11.7	73
8	Nearest Insertion	Simulated Annealing	100	953.06	806.23	15.4	69
9	Farthest Neighbor	Tanpa improvement	100	7793.98	7793.98	0.0	2
10	Farthest Neighbor	2-opt	100	7793.98	840.00	89.2	10
11	Farthest Neighbor	Tabu Search	100	7793.98	833.50	89.3	63
12	**Farthest Neighbor**	**Simulated Annealing**	**100**	**7793.98**	**803.32**	**89.7**	**44**
13	Arbitrary (acak)	Tanpa improvement	100	5364.69	5364.69	0.0	1
14	Arbitrary (acak)	2-opt	100	5364.69	930.66	82.7	10
15	Arbitrary (acak)	Tabu Search	100	5364.69	891.44	83.4	64
16	Arbitrary (acak)	Simulated Annealing	100	5364.69	804.28	85.0	74

Lampiran 11. Tampilan Perbandingan Hasil Run pada 100 Titik Seed 1

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	100	998.53	998.53	0.0	3
2	Nearest Neighbor	2-opt	100	998.53	812.31	18.6	5
3	Nearest Neighbor	Tabu Search	100	998.53	812.03	18.7	46
4	Nearest Neighbor	Simulated Annealing	100	998.53	819.97	17.9	69
5	Nearest Insertion	Tanpa improvement	100	958.87	958.87	0.0	8
6	Nearest Insertion	2-opt	100	958.87	883.19	7.9	8
7	Nearest Insertion	Tabu Search	100	958.87	857.87	10.5	69
8	**Nearest Insertion**	**Simulated Annealing**	**100**	**958.87**	**804.68**	**16.1**	**63**
9	Farthest Neighbor	Tanpa improvement	100	7740.07	7740.07	0.0	3
10	Farthest Neighbor	2-opt	100	7740.07	848.56	89.0	19
11	Farthest Neighbor	Tabu Search	100	7740.07	846.12	89.1	62
12	Farthest Neighbor	Simulated Annealing	100	7740.07	822.90	89.4	48
13	Arbitrary (acak)	Tanpa improvement	100	5139.25	5139.25	0.0	0
14	Arbitrary (acak)	2-opt	100	5139.25	837.25	83.7	14
15	Arbitrary (acak)	Tabu Search	100	5139.25	837.25	83.7	58
16	Arbitrary (acak)	Simulated Annealing	100	5139.25	849.92	83.5	70

Lampiran 12. Tampilan Perbandingan Hasil Run pada 100 Titik Seed 2

VII PERBANDINGAN RUN

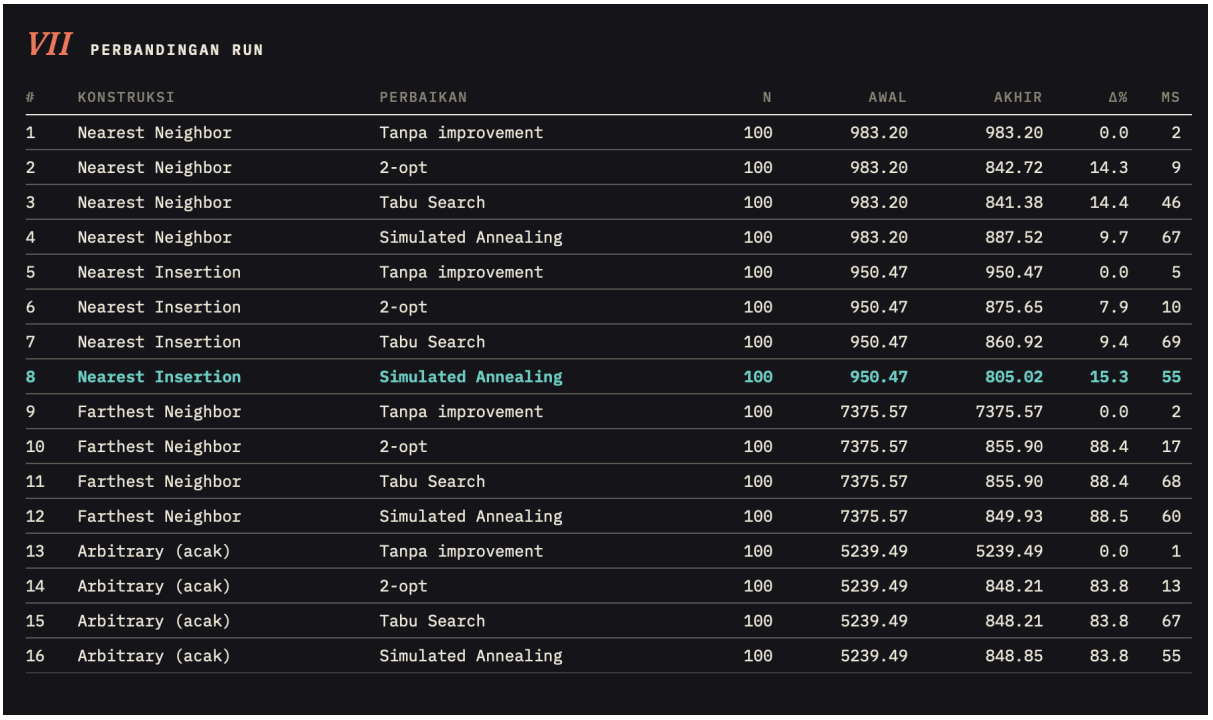
#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	100	1097.62	1097.62	0.0	2
2	Nearest Neighbor	2-opt	100	1097.62	835.47	23.9	9
3	Nearest Neighbor	Tabu Search	100	1097.62	835.47	23.9	50
4	**Nearest Neighbor**	**Simulated Annealing**	**100**	**1097.62**	**790.12**	**28.0**	**62**
5	Nearest Insertion	Tanpa improvement	100	903.26	903.26	0.0	7
6	Nearest Insertion	2-opt	100	903.26	858.30	5.0	9
7	Nearest Insertion	Tabu Search	100	903.26	843.08	6.7	59
8	Nearest Insertion	Simulated Annealing	100	903.26	835.86	7.5	52
9	Farthest Neighbor	Tanpa improvement	100	7462.17	7462.17	0.0	1
10	Farthest Neighbor	2-opt	100	7462.17	862.45	88.4	16
11	Farthest Neighbor	Tabu Search	100	7462.17	860.51	88.5	67
12	Farthest Neighbor	Simulated Annealing	100	7462.17	823.53	89.0	64
13	Arbitrary (acak)	Tanpa improvement	100	4799.27	4799.27	0.0	1
14	Arbitrary (acak)	2-opt	100	4799.27	875.38	81.8	12
15	Arbitrary (acak)	Tabu Search	100	4799.27	875.38	81.8	69
16	Arbitrary (acak)	Simulated Annealing	100	4799.27	831.10	82.7	62

Lampiran 13. Tampilan Perbandingan Hasil Run pada 100 Titik Seed 3

VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	100	938.82	938.82	0.0	2
2	**Nearest Neighbor**	**2-opt**	**100**	**938.82**	**819.74**	**12.7**	**4**
3	**Nearest Neighbor**	**Tabu Search**	**100**	**938.82**	**819.74**	**12.7**	**56**
4	Nearest Neighbor	Simulated Annealing	100	938.82	834.28	11.1	75
5	Nearest Insertion	Tanpa improvement	100	1023.39	1023.39	0.0	7
6	Nearest Insertion	2-opt	100	1023.39	917.64	10.3	9
7	Nearest Insertion	Tabu Search	100	1023.39	914.75	10.6	57
8	Nearest Insertion	Simulated Annealing	100	1023.39	842.58	17.7	68
9	Farthest Neighbor	Tanpa improvement	100	7435.33	7435.33	0.0	1
10	Farthest Neighbor	2-opt	100	7435.33	891.04	88.0	9
11	Farthest Neighbor	Tabu Search	100	7435.33	872.78	88.3	59
12	Farthest Neighbor	Simulated Annealing	100	7435.33	834.96	88.8	59
13	Arbitrary (acak)	Tanpa improvement	100	5519.59	5519.59	0.0	0
14	Arbitrary (acak)	2-opt	100	5519.59	868.67	84.3	10
15	Arbitrary (acak)	Tabu Search	100	5519.59	868.67	84.3	51
16	Arbitrary (acak)	Simulated Annealing	100	5519.59	866.33	84.3	61

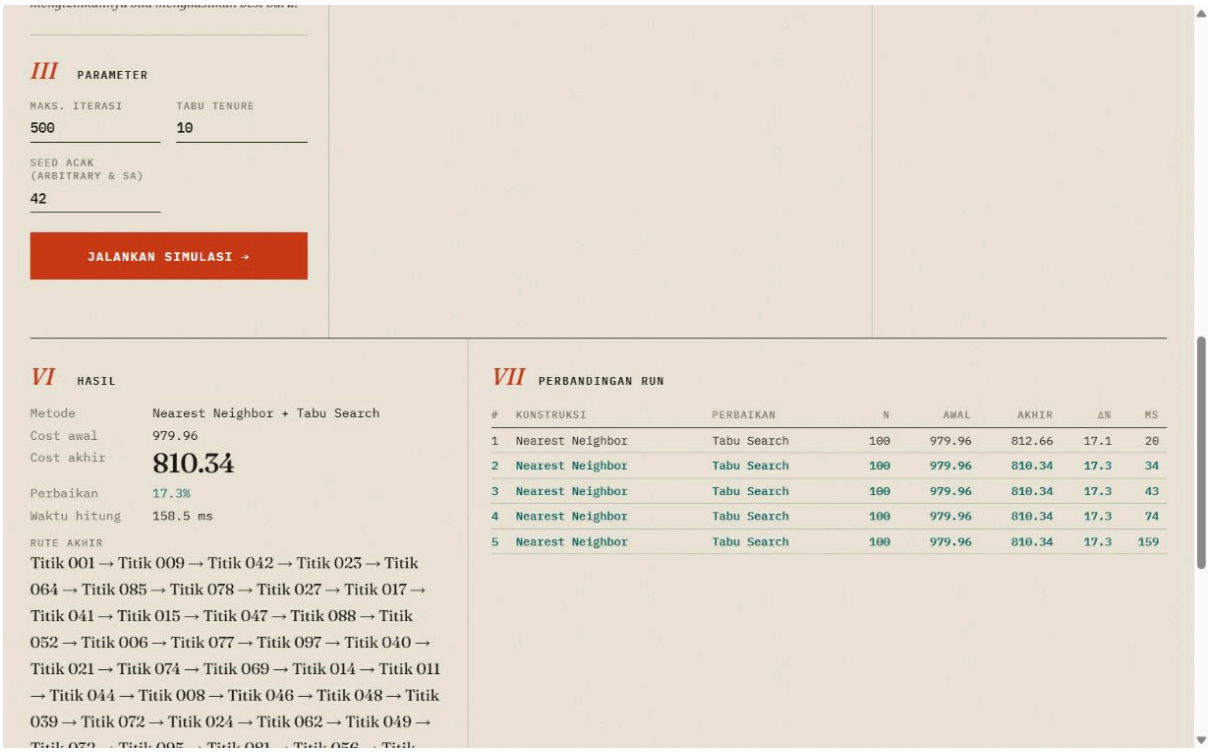
Lampiran 14. Tampilan Perbandingan Hasil Run pada 100 Titik Seed 4



VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tanpa improvement	100	983.20	983.20	0.0	2
2	Nearest Neighbor	2-opt	100	983.20	842.72	14.3	9
3	Nearest Neighbor	Tabu Search	100	983.20	841.38	14.4	46
4	Nearest Neighbor	Simulated Annealing	100	983.20	887.52	9.7	67
5	Nearest Insertion	Tanpa improvement	100	950.47	950.47	0.0	5
6	Nearest Insertion	2-opt	100	950.47	875.65	7.9	10
7	Nearest Insertion	Tabu Search	100	950.47	860.92	9.4	69
8	**Nearest Insertion**	**Simulated Annealing**	**100**	**950.47**	**805.02**	**15.3**	**55**
9	Farthest Neighbor	Tanpa improvement	100	7375.57	7375.57	0.0	2
10	Farthest Neighbor	2-opt	100	7375.57	855.90	88.4	17
11	Farthest Neighbor	Tabu Search	100	7375.57	855.90	88.4	68
12	Farthest Neighbor	Simulated Annealing	100	7375.57	849.93	88.5	60
13	Arbitrary (acak)	Tanpa improvement	100	5239.49	5239.49	0.0	1
14	Arbitrary (acak)	2-opt	100	5239.49	848.21	83.8	13
15	Arbitrary (acak)	Tabu Search	100	5239.49	848.21	83.8	67
16	Arbitrary (acak)	Simulated Annealing	100	5239.49	848.85	83.8	55

Lampiran 15. Tampilan Perbandingan Hasil Run pada 100 Titik Seed 5



III PARAMETER

MAKS. ITERASI: 500

TABU TENURE: 10

SEED ACAK (ARBITRARY & SA): 42

JALANKAN SIMULASI →

VI HASIL

Metode: Nearest Neighbor + Tabu Search
Cost awal: 979.96
Cost akhir: 810.34
Perbaikan: 17.3%
Waktu hitung: 158.5 ms

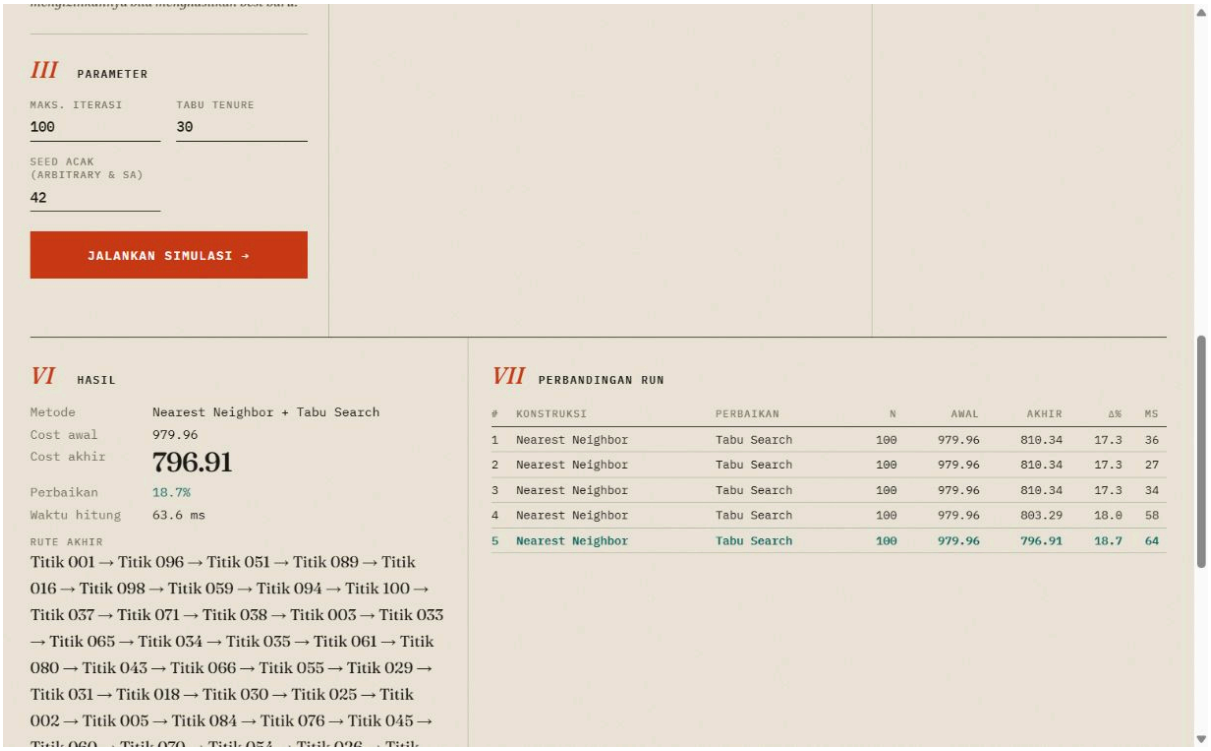
RUTE AKHIR

Titik 001 → Titik 009 → Titik 042 → Titik 023 → Titik 064 → Titik 085 → Titik 078 → Titik 027 → Titik 017 → Titik 041 → Titik 015 → Titik 047 → Titik 088 → Titik 052 → Titik 006 → Titik 077 → Titik 097 → Titik 040 → Titik 021 → Titik 074 → Titik 069 → Titik 014 → Titik 011 → Titik 044 → Titik 008 → Titik 046 → Titik 048 → Titik 039 → Titik 072 → Titik 024 → Titik 062 → Titik 049 →

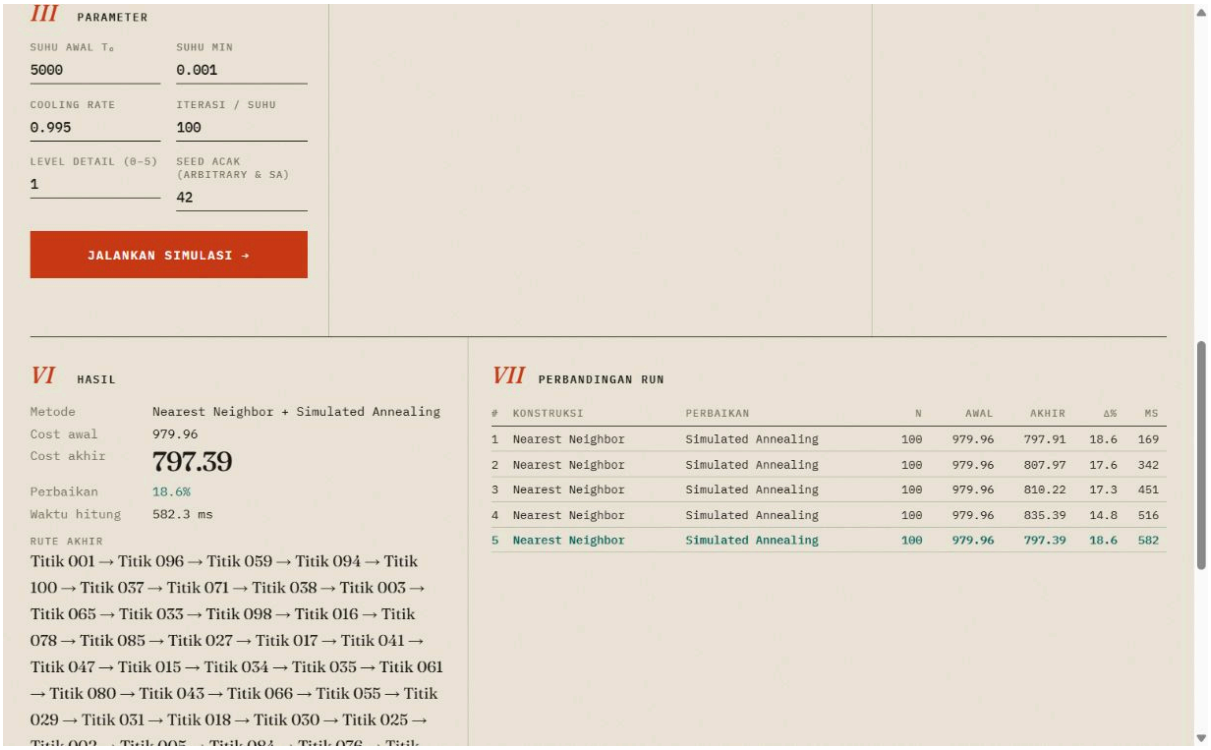
VII PERBANDINGAN RUN

#	KONSTRUKSI	PERBAIKAN	N	AWAL	AKHIR	Δ%	MS
1	Nearest Neighbor	Tabu Search	100	979.96	812.66	17.1	20
2	Nearest Neighbor	Tabu Search	100	979.96	810.34	17.3	34
3	Nearest Neighbor	Tabu Search	100	979.96	810.34	17.3	43
4	Nearest Neighbor	Tabu Search	100	979.96	810.34	17.3	74
5	Nearest Neighbor	Tabu Search	100	979.96	810.34	17.3	159

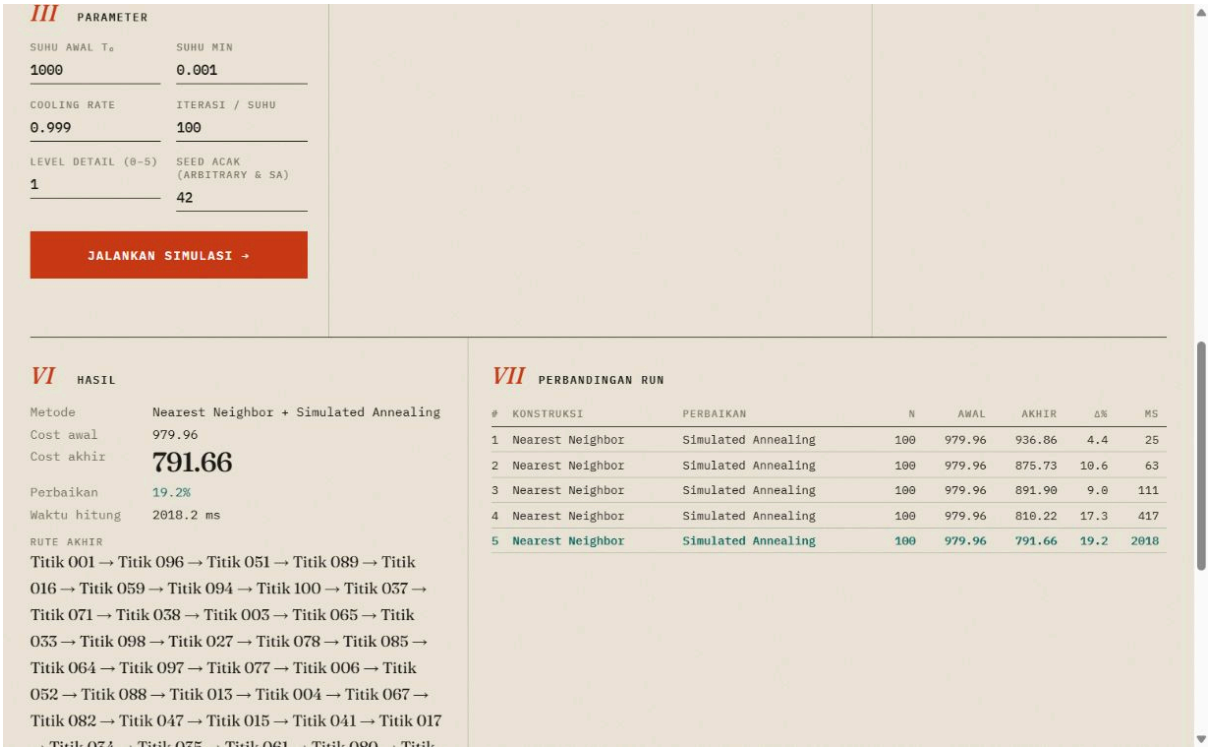
Lampiran 16. Tampilan Perbandingan Hasil Run Perubahan Maksimum Iterasi Tabu Search



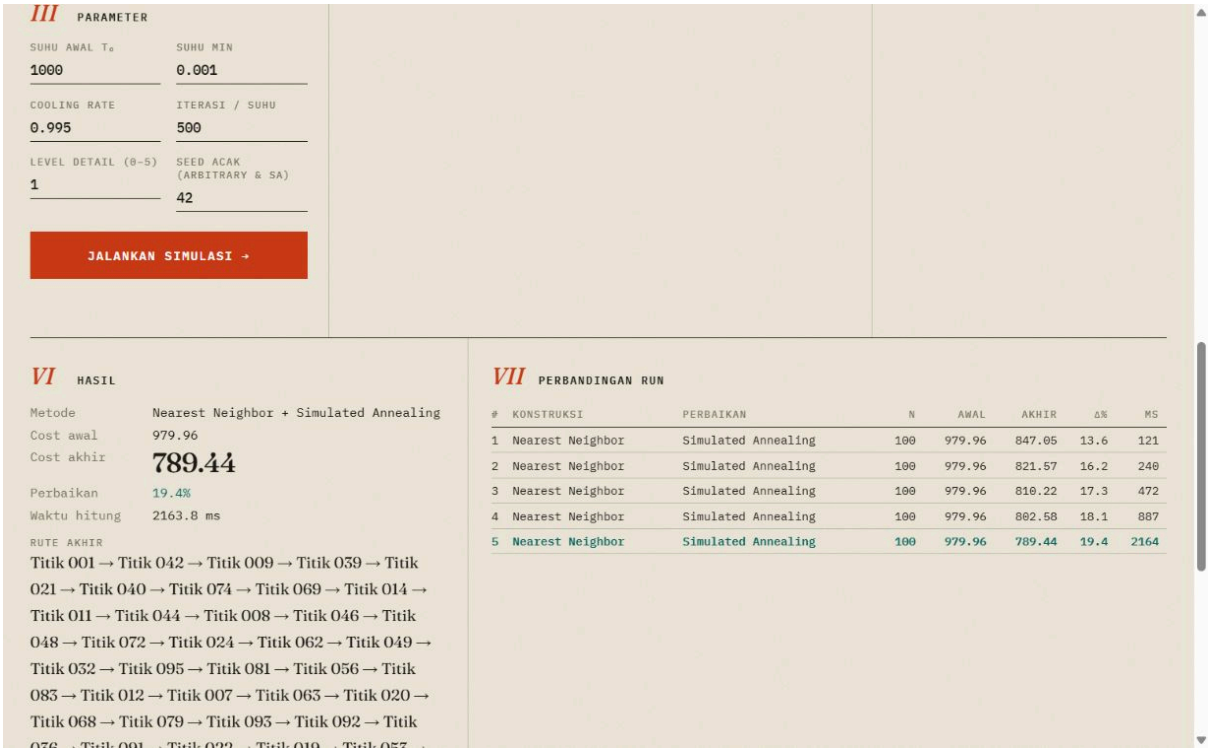
Lampiran 17. Tampilan Perbandingan Hasil Run Perubahan Tabu Tenure Tabu Search



Lampiran 18. Tampilan Perbandingan Hasil Run Perubahan Temperatur Awal Simulated Annealing



Lampiran 19. Tampilan Perbandingan Hasil Run Perubahan Cooling Rate Simulated Annealing



Lampiran 20. Tampilan Perbandingan Hasil Run Perubahan Iterasi per Temperatur Simulated Annealing